Master Blueprint: Production Hybrid RAG Engine Architecture

This document is your **reusable architectural playbook**. Keep this as your blueprint whenever you build a RAG system from scratch.

System Map: What Each File Does

```
preparation/

 docker-compose.yml                  # 1. INFRASTRUCTURE: Runs Qdrant Vector DB container

 src/preparation/
    config.py                        # 2. SETTINGS: Single source of truth for hosts, ports, dims

    processing/
       chunker.py                    # 3. TEXT CHUNKER: Sliding-window with stride math & char offsets
       markdown_chunker.py           # 4. MARKDOWN CHUNKER: Structure-aware Buffer & Flush by # headers
       embedder.py                   # 5. EMBEDDER: Converts text to 384-dim ONNX vectors via FastEmbed

    storage/
        qdrant.py                    # 6. STORAGE ADAPTER: DB connection, upsert, & multi-tenant filter

 app/
     schemas.py                      # 7. CONTRACTS: Pydantic models for API request/response validation
     main.py                         # 8. ORCHESTRATOR: FastAPI server connecting chunker, embedder, DB
```

File 1: `docker-compose.yml` (The Database Engine)

Purpose

To spin up the Qdrant Vector Database in an isolated container without needing manual system installations. It ensures data persists on your laptop's disk even if the container restarts.

Key Lines & Why We Wrote Them

```
services:
  qdrant:
    image: qdrant/qdrant:latest
    ports:
      - "6333:6333" # Port 6333: REST API & Web Dashboard (used by our Python client and browser UI)
      - "6334:6334" # Port 6334: High-speed gRPC interface (for ultra-low-latency production search)
    volumes:
      - ./qdrant_storage:/qdrant/storage # Maps container storage to local folder so vectors don't vanish on r
```

File 2: `src/preparation/config.py` (The Single Source of Truth)

Purpose

Centralizes every magic string, port number, model name, and dimension. If your Qdrant port changes or you switch models, you update **one line** here instead of editing 10 different files.

Key Lines & Why We Wrote Them

```
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    # Where Qdrant lives
    qdrant_host: str = "localhost"
    qdrant_port: int = 6333
    qdrant_grpc_port: int = 6334

    # What collection (table) we store vectors in
    collection_name: str = "rag_knowledge_base"

    # Embedding model specification (BGE-Small produces 384-length float vectors)
    embedding_model: str = "BAAI/bge-small-en-v1.5"
    embedding_dim: int = 384

    # Default chunking parameters
    chunk_size: int = 500
    chunk_overlap: int = 100

    # Reads from .env file automatically if present
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

# Singleton Pattern: Create one instance that the entire app shares
settings = Settings()
```

File 3: `src/preparation/processing/chunker.py` (Sliding Window Chunker)

Purpose

Breaks large unstructured plain text into manageable blocks that fit into the embedding models context limit, while using overlapping windows so sentences aren't chopped in half.

Key Lines & Why We Wrote Them

```
@dataclass
class chunk:
    chunk_id: str
    text: str
    start_char: int   # Exact character where chunk starts in original document
    end_char: int     # Exact character where chunk ends in original document

class SlidingWindowChunker:
    def __init__(self, chunk_size: int | None = None, chunk_overlap: int | None = None):
        self.chunk_size = chunk_size or settings.chunk_size
        self.chunk_overlap = chunk_overlap or settings.chunk_overlap

        # GUARD CLAUSE: Overlap cannot be >= size, or stride will be <= 0 (infinite loop!)
        if self.chunk_overlap >= self.chunk_size:
            raise ValueError("Chunk overlap must be smaller than chunk size")

    def chunk_text(self, text: str) -> list[chunk]:
        if not text.strip(): return []

        # Stride formula: How many characters to jump forward for the next window
        step = self.chunk_size - self.chunk_overlap
        start = 0
        chunks = []
```

```
        while start < len(text):
            chunk_text = text[start : start + self.chunk_size]
            end_char = start + len(chunk_text)

            chunks.append(chunk(
                chunk_id=str(uuid.uuid4()),
                text=chunk_text,
                start_char=start,
                end_char=end_char
            ))

            if end_char >= len(text):
                break # Reached the end of text
            start += step # Advance the sliding window

        return chunks
```

File 4: `src/preparation/processing/markdown_chunker.py` (Structure-Aware Chunker)

Purpose

Preserves document hierarchy. Instead of arbitrarily slicing sentences, it splits by Markdown headers (`#`, `##`, `###`) using the **Buffer & Flush / Accumulator pattern**, tagging each chunk with its breadcrumb header.

Key Lines & Why We Wrote Them

```
class MarkdownChunk(BaseModel):
    chunk_id: str
    text: str
    header: str    # e.g., "Architecture" or "Multi-Tenancy"
    level: int     # Depth: 1 for #, 2 for ##, 3 for ###

class MarkdownHeaderChunker:
    def chunk_markdown(self, markdown_text: str) -> list[MarkdownChunk]:
        chunks: list[MarkdownChunk] = []
        lines = markdown_text.splitlines()

        current_header = "Introduction"
        current_level = 1
        current_lines: list[str] = [] # The Accumulator Buffer

        # FLUSH HELPER: Packs accumulated lines into a chunk and clears the buffer
        def flush_section():
            text_block = "\n".join(current_lines).strip()
            if not text_block: return

            # If section fits under max_size, keep it whole
            if len(text_block) <= self.max_chunk_size:
                chunks.append(MarkdownChunk(
                    chunk_id=str(uuid.uuid4()),
                    text=text_block,
                    header=current_header,
                    level=current_level
                ))
            else:
```

```
                # If section is massive, fallback to sliding window to avoid token overflow
                for sc in self.fallback_chunker.chunk_text(text_block):
                    chunks.append(MarkdownChunk(
                        chunk_id=str(uuid.uuid4()),
                        text=sc.text,
                        header=current_header,
                        level=current_level
                    ))

        # LOOP THROUGH LINES
        for line in lines:
            stripped = line.strip()
            if stripped.startswith("#"):
                flush_section()          # 1. Flush previous accumulated section
                current_lines.clear()    # 2. Reset buffer

                # Count consecutive '#' symbols to determine heading depth
                hash_count = len(stripped) - len(stripped.lstrip("#"))
                current_level = hash_count
                current_header = stripped.lstrip("#").strip()
            else:
                current_lines.append(line) # Normal paragraph line -> accumulate

        flush_section() # Final flush for trailing content at end of file
        return chunks
```

File 5: `src/preparation/processing/embedder.py` (The Mathematical Vectorizer)

Purpose

Converts raw strings into mathematical vectors (arrays of 384 floating-point numbers). We use a class so the ONNX model is loaded into RAM **once** upon initialization rather than re-loaded on every query.

Key Lines & Why We Wrote Them

```
from fastembed import TextEmbedding

class LocalEmbedder:
    def __init__(self, model_name: str | None = None):
        # Loads ONNX runtime weights once in RAM (~67MB)
        self.model = TextEmbedding(model_name=self.model_name)

    def embed_chunks(self, texts: list[str]) -> list[list[float]]:
        # FastEmbed returns a Python generator; we convert to list of float lists
        embeddings_generator = self.model.embed(texts)
        return [emb.tolist() for emb in embeddings_generator]

    def embed_query(self, query: str) -> list[float]:
        # FastEmbed has an optimized embed_query for search queries
        query_generator = self.model.embed_query(query)
        return next(query_generator).tolist()
```

File 6: `src/preparation/storage/qdrant.py` (Database Adapter & Multi-Tenancy)

Purpose

Talks directly to Qdrant. It manages collection creation, idempotent upserts (preventing duplicate points), and executes **single-stage multi-tenant payload filtering**.

Key Lines & Why We Wrote Them

```
class QdrantStorage:
    def __init__(self):
        # Persistent client connection
        self.client = QdrantClient(host=settings.qdrant_host, port=settings.qdrant_port)

    def ensure_collection_exists(self, collection_name: str | None = None) -> bool:
        # Idempotent: checks before creating to avoid 409 Conflict errors
        if self.client.collection_exists(collection_name=target):
            return False
        self.client.create_collection(
            collection_name=target,
            vectors_config=models.VectorParams(
                size=settings.embedding_dim,       # Must match embedder output (384)
                distance=models.Distance.COSINE   # Cosine similarity for semantic search
            )
        )
        return True

    def upsert_chunks(self, chunks: list, vectors: list[list[float]], tenant_id: str | None = None) -> int:
        points = []
        for chunk, vector in zip(chunks, vectors):
            # Deterministic UUID5: Same chunk_id always generates the same point UUID!
            point_id = str(uuid.uuid5(uuid.NAMESPACE_DNS, chunk.chunk_id))

            # getattr() ensures compatibility with BOTH SlidingWindow and Markdown chunks
            point = models.PointStruct(
                id=point_id,
                vector=vector,
                payload={
                    "tenant_id": tenant_id, # Tenant tag for zero-leak security
                    "text": chunk.text,
                    "header": getattr(chunk, "header", None),
                    "level": getattr(chunk, "level", None),
                    "start_char": getattr(chunk, "start_char", None),
                    "end_char": getattr(chunk, "end_char", None)
                }
            )
            points.append(point)
        self.client.upsert(collection_name=target, points=points)
        return len(points)

    def search(self, query_vector: list[float], limit: int = 3, tenant_id: str | None = None):
        # ENTERPRISE MULTI-TENANCY:
        # Pre-filters data in the vector index so company A CANNOT see company B's vectors
        query_filter = None
        if tenant_id:
            query_filter = models.Filter(
                must=[
                    models.FieldCondition(
                        key="tenant_id",
                        match=models.MatchValue(value=tenant_id)
                    )
```

```
                ]
            )

        results = self.client.query_points(
            collection_name=target,
            query=query_vector,
            query_filter=query_filter,
            limit=limit
        )
        return results.points
```

File 7: `app/schemas.py` (API Contracts / Pydantic Models)

Purpose

Validates and type-checks all incoming HTTP requests and outgoing HTTP responses. If a client sends invalid data (e.g. text that is too short, or a negative limit), FastAPI rejects it with a clear 422 error before it touches our database.

Key Lines & Why We Wrote Them

```
class IngestionRequest(BaseModel):
    doc_type: str = "text" # "text" or "markdown"
    document_id: str
    tenant_id: str = "default_tenant"
    context: str = Field(..., min_length=10, description='Raw text documents')
    chunk_size: int = Field(default=300, gt=0)
    chunk_overlap: int = Field(default=40, ge=0)

class IngestionResponse(BaseModel):
    document_id: str
    tenant_id: str
    total_chunks: int
    collection: str
    status: str = "success"

class search(BaseModel):
    query: str = Field(..., min_length=2)
    tenant_id: str | None = None
    limit: int = Field(default=3, gt=0, le=20) # Prevents requesting 10,000 items and crashing server

class ResultItem(BaseModel):
    id: str
    score: float
    text: str
    tenant_id: str | None = None
    header: str | None = None # Breadcrumb header from markdown

class searchresponse(BaseModel):
    query: str
    total_found: int
    results: list[ResultItem]
```

File 8: `app/main.py` (The API Orchestrator)

Purpose

Exposes HTTP REST endpoints. It wires together the request, picks the chunker, converts chunks to embeddings, saves to Qdrant, and returns formatted responses.

Key Lines & Why We Wrote Them

```
# Lifespan: Runs on server boot to ensure DB tables exist before taking requests
@asynccontextmanager
async def lifespan(app: FastAPI):
    storage.ensure_collection_exists()
    yield

app = FastAPI(title="Production Rag Engine", lifespan=lifespan)

# Health Check
@app.get("/health")
def health_check():
    return {"status": "healthy", "qdrant_alive": storage.health()}

# Ingestion Route: Dynamic chunker selection
@app.post("/api/v1/documents/ingest", status_code=status.HTTP_201_CREATED, response_model=IngestionResponse)
def ingest_document(payload: IngestionRequest):
    # 1. Choose Chunker
    if payload.doc_type == "markdown":
        chunker = MarkdownHeaderChunker(max_chunk_size=payload.chunk_size, chunk_overlap=payload.chunk_overlap
        chunks = chunker.chunk_markdown(payload.context)
    else:
        chunker = SlidingWindowChunker(chunk_size=payload.chunk_size, chunk_overlap=payload.chunk_overlap)
        chunks = chunker.chunk_text(payload.context)

    # 2. Embed
    texts = [c.text for c in chunks]
    vectors = embedder.embed_chunks(texts)

    # 3. Store
    total_stored = storage.upsert_chunks(chunks, vectors, tenant_id=payload.tenant_id)

    # 4. Respond
    return IngestionResponse(
        document_id=payload.document_id,
        tenant_id=payload.tenant_id,
        total_chunks=total_stored,
        collection=settings.collection_name,
        status="success"
    )

# Search Route
@app.post("/api/v1/search", response_model=searchresponse)
def search_documents(payload: search):
    # 1. Embed query into vector
    query_vector = embedder.embed_query(payload.query)

    # 2. Query Qdrant with tenant filter
    raw_results = storage.search(query_vector=query_vector, limit=payload.limit, tenant_id=payload.tenant_id)

    # 3. Format response
    items = [
        ResultItem(
```

```
            id=str(pt.id),
            score=round(pt.score, 4),
            text=pt.payload.get("text", ""),
            tenant_id=pt.payload.get("tenant_id"),
            header=pt.payload.get("header")
        )
        for pt in raw_results
    ]

    return searchresponse(query=payload.query, total_found=len(items), results=items)
```

The 5-Step Replication Checklist for Project #2

When you open an empty folder for your next project, follow this exact sequence:

- 1. **`uv init` & `docker-compose.yml`**: Define your vector DB image and host storage path.
- 2. **`config.py`**: Create your `Settings(BaseSettings)` class for host, ports, dims, and collection name.
- 3. **`processing/`**:
- Write your `chunker.py` (Sliding window stride logic).
- Write your `markdown_chunker.py` (Buffer & flush by header).
- Write your `embedder.py` (Load ONNX model once, `embed_chunks` + `embed_query`).
- 4. **`storage/`**: Write your DB client (`ensure_collection_exists`, `upsert_chunks` with UUID5, and `search` with multi-tenant filter).
- 5. **`app/`**:
- Define `schemas.py` with Pydantic request/response models.
- Write `main.py` with lifespan startup and the two REST routes (`/ingest` and `/search`).